

Multi-Tenant SaaS Platform Plan

Contoso Outdoors

Azure Marketplace SaaS Transactable Offer
August 2026

1. Platform Requirements Summary

Dimension	Current State	Target State
Tenant definition	None — single-tenant, single catalogue	One tenant = one retail brand/storefront
Compute	Single App Service B1	Shared App Service (pooled), scale to stamps
Data	Static JSON files on disk	Tenant-isolated (Cosmos DB or Azure SQL)
Identity	System-assigned managed identity only	Entra ID B2C for end-users + Entra ID for admins
AI/Chat	Chat API stubs (grounded, visual, promptflow)	Per-tenant chat config with usage metering
Marketplace	Not listed	SaaS Transactable on Azure Marketplace
Regions	Single region	Single-region starter → multi-region
Compliance	None specified	SOC 2 readiness recommended

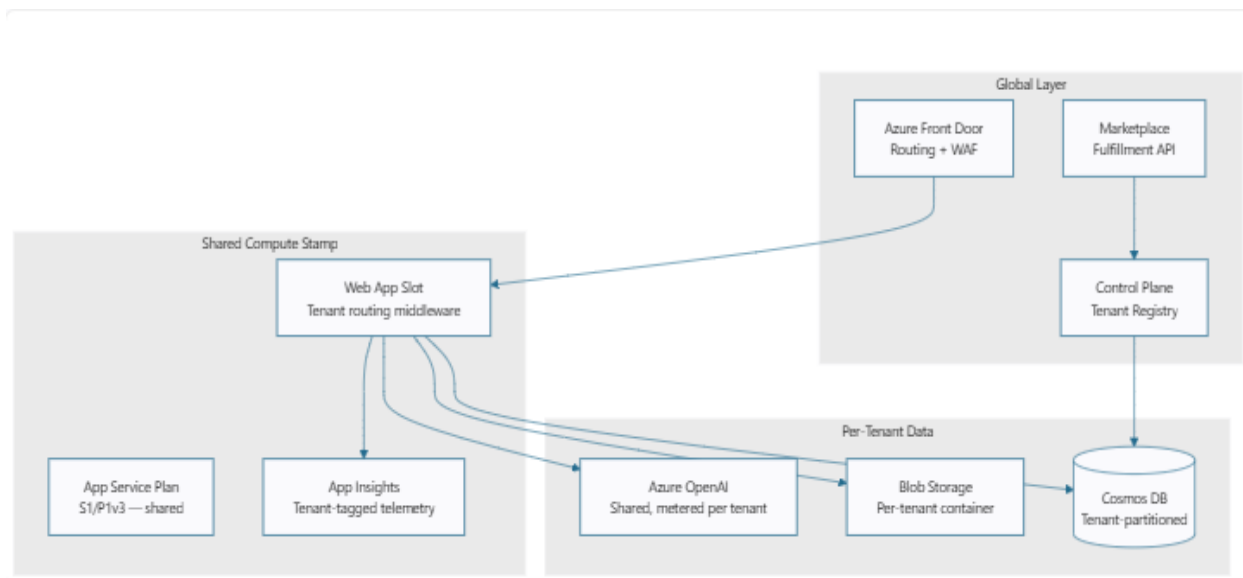
2. Tenant Definition & Business Mapping

A tenant is one retail brand operating a storefront through Contoso Outdoors.

Concept	Maps To
Tenant	A retail brand/company (e.g., Alpine Gear Co.)
Users	Storefront visitors (anonymous or authenticated shoppers)
Tenant admin	Brand owner who manages catalogue, branding, chat config
Subscription	Marketplace purchase → creates tenant with a plan
Environment	Not per-tenant; managed by ISV (dev/staging/prod)

3. Multitenancy Decision Record

Isolation Model: Shared Infrastructure / Isolated Data



Layer	Model	Rationale
Compute	Shared (pooled)	Low cost; App Service handles routing; rate limiting mitigates noisy neighbor
Data	Isolated (partition key)	Cosmos DB with tenantId partition key — logical isolation, shared throughput
Static assets	Isolated (per-tenant container)	Product images, manuals, branding assets separated per tenant
Identity	Shared Entra ID B2C	Tenant admin mapped via custom claims; shoppers scoped per tenant
AI/Chat	Shared Azure OpenAI	Metered per tenant; system prompt customized per catalog
Networking	Shared Front Door	Custom domain mapping per tenant; shared WAF rules

Tenant Context Propagation

1. Request arrives at Front Door — Host header resolves to tenant
2. App Service middleware validates tenant, injects tenantId into request context
3. API/page handler reads tenantId, queries tenant-scoped data from Cosmos DB
4. App Insights logs with tenantId as custom dimension

Tenant ID is derived from authenticated session or host header, never trusted from client input alone. Background jobs receive tenant ID via message metadata, re-validated before processing.

4. Staged Execution Plan

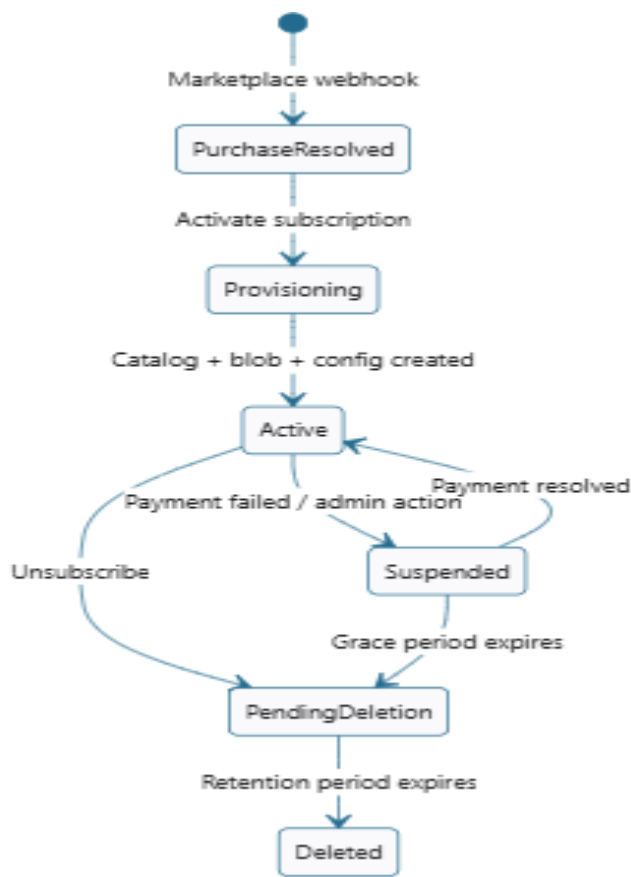
Stage 1: ISV Landing Zone Foundation

Task	Detail
Subscription layout	Platform subscription (shared services) + Workload subscription (SaaS app)
Management groups	ISV root → Platform, Workloads (dev/prod)
RBAC	Least-privilege roles for CI/CD, operators, tenant admins
Policy	Enforce HTTPS, TLS 1.2, tagging, allowed regions
Networking	Front Door → App Service VNet integration → Private endpoints
Monitoring baseline	Log Analytics + App Insights (promote existing setup)

Stage 2a: Tenant Isolation Implementation

- Replace static JSON catalogue with Cosmos DB (partition key: tenantId)
- Create Blob Storage with per-tenant containers for images/manuals
- Add tenant routing middleware to Next.js (resolve host → tenantId)
- Implement tenant-scoped API authorization on all chat and product endpoints

Stage 2b: Tenant Lifecycle Automation



State	Automation
PurchaseResolved	Fulfillment API resolves token, creates tenant record
Provisioning	Create Cosmos DB tenant partition, blob container, seed config
Active	Tenant fully operational, metering active
Suspended	Read-only access, metering paused
PendingDeletion	30-day data retention, then purge

Stage 2c: Security & Compliance

Control	Implementation
Tenant-context validation	Middleware derives tenantId from auth token; never from query param
Cross-tenant isolation testing	Automated tests: Tenant A cannot read Tenant B data
Secret management	Key Vault per environment; connection strings via managed identity
Data encryption	Cosmos DB SSE (platform default) + TLS in transit
Audit logging	All tenant lifecycle events + admin actions to Log Analytics

Stage 3: SaaS Accelerator & Marketplace

3a: SaaS Accelerator Bootstrap

Component	Purpose
Landing page (/marketplace/activate)	Resolves purchase token, shows plan details, activates subscription
Webhook (/api/marketplace/webhook)	Handles ChangePlan, Suspend, Reinstate, Unsubscribe
Admin portal (/admin)	Tenant management, plan overview, usage dashboard

3b: Fulfillment & Metering

Meter Dimension	Unit	Billing
ai-chat-queries	Per 1,000 requests	Overage above plan inclusion
product-catalogue-size	Per 100 products	Tiered by plan
monthly-storefront-visitors	Per 10,000 unique visitors	Overage above plan cap

Pricing Plans

Plan	Base Price	Included	Overage
Starter	\$49/mo	1 store, 100 products, 1K chat queries	\$0.01/query, \$5/100 products
Pro	\$199/mo	5 stores, 1K products, 10K chat queries	\$0.008/query, \$3/100 products
Enterprise	Custom	Unlimited stores, custom SLA	Negotiated

3c: Marketplace Onboarding

Gate	Status
Partner Centre account	☐ Not started
Publisher profile + legal	☐ Not started
Offer created (SaaS Transactable)	☐ Not started
Plans + pricing configured	☐ Not started
Technical config (landing page + webhook)	☐ Not started
Private preview testing	☐ Not started
Certification submission	☐ Not started
Go-live	☐ Not started

Stage 4: Scale & SRE

Concern	Approach
SLO	99.9% availability, p95 response < 500ms
Noisy neighbor	Per-tenant rate limiting on chat API; Cosmos DB RU budgets
Scaling	App Service autoscale (CPU/memory); Cosmos DB autoscale RU
Deployment stamps	Not needed initially; trigger at ~200 tenants or regional expansion
Observability	App Insights with tenantId custom dimension; per-tenant dashboards
Cost allocation	Tag resources + Cosmos DB RU consumption attributed per tenant
DR	Cosmos DB multi-region (when justified); deployment slots for zero-downtime

5. Deployment Blueprint

Track: Single-Region Starter SaaS

Phase	Timeline	Deliverables
Phase 1 — Foundation	Week 1–2	Resource Group, Front Door + WAF, App Service Plan (S1), Key Vault, Log Analytics + App Insights
Phase 2 — Data Layer	Week 3–4	Cosmos DB (serverless/autoscale), Blob Storage + CDN, Tenant registry collection, Data migration: JSON → Cosmos DB
Phase 3 — Multi-Tenant App	Week 5–8	Tenant routing middleware, Per-tenant catalogue API, Per-tenant chat API (metered), Tenant admin portal, Isolation tests
Phase 4 — Marketplace	Week 9–12	Fulfillment API integration, Landing page + webhook, Metering pipeline, Partner Centre submission, Private preview → Go-live

6. Offer Strategy Decision

Alternative	Verdict	Rationale
SaaS Transactable	✅ Selected	ISV-hosted, Microsoft-billed, co-sell eligible
Azure Managed Application	❌ Rejected	Single tenant deploys in customer sub, wrong model
Container offer	❌ Rejected	Customer-hosted, operational burden on customer
VM offer	❌ Rejected	Not applicable to web SaaS

7. Marketplace Readiness Checklist

- ☐ Entra ID app registration for Marketplace auth
- ☐ Fulfillment API v2 webhook endpoint deployed and reachable
- ☐ Landing page resolves purchase token and activates subscription
- ☐ Metered billing emits usage events within 24 hours
- ☐ Cross-tenant data isolation verified by automated tests
- ☐ Privacy policy and terms of use URLs configured
- ☐ Support contact and help documentation published
- ☐ Private preview tested with at least 2 pilot tenants
- ☐ Partner Centre offer passes certification validation

8. Day-2 Operations & SRE Control Plan

Area	Controls
Tenant onboarding	Automated provisioning via Marketplace webhook; idempotent; < 60s SLA
Tenant offboarding	30-day soft-delete; data export available; purge after retention
Incident response	PagerDuty/on-call; per-tenant blast radius assessment; customer comms
Deployment	Blue/green via App Service slots; canary via Front Door traffic splitting
Capacity planning	Monthly tenant density review; stamp scale-out trigger at ~200 tenants
Cost management	Per-tenant cost attribution; monthly margin analysis; plan pricing review
Compliance	Quarterly SOC 2 evidence collection; annual penetration testing